

音视频模型

GLM-Realtime

 Copy page

概览

GLM-Realtime 是一款音视频通话模型，能够提供实时的视频通话功能，通话记忆时长长达2分钟，具有跨文本、音频和视频进行实时推理的能力。



输入模式

视频、音频、文本



输出模式

音频



最大输出 Tokens

1K



价格



Show GLM-Realtime-Flash



Show GLM-Realtime-Air



上下文窗口



Show 音频通话



Show 视频通话

推荐场景

▼ 口语陪练



通过实时对话+视频反馈，及时纠正用户发音错误，支持视频捕捉用户表情、识别物体、浏览文档。



▸ 实时翻译

▸ 面试模拟

▸ 旅行导游

使用资源

音视频实时 API 构建在 WebSocket API 之上，通过集成 Realtime API 或 SDK, 参考开源仓库样例代码，快速接入成服务。



Realtime SDK

Realtime Python
Golang TypeScript
SDK



前端样例代码

Realtime API 的使用场
景前端样例代码

↑ 详细介绍



GLM-Realtime 通过流式推理降低视频通话延时，AI可以进行流畅的通话，人也可以实时打断AI。除了实时音频交互外，GLM-Realtime 还可通过手机或AIPC的摄像头与人互动，通过共享电脑屏幕阅读页面信息，通过视频流理解对话当前的环境。

在语音交互方面，GLM-Realtime 创新性地实现了清唱功能，首次让大模型具备在对话中的歌唱能力。

同时，我们将 GLM-Realtime API 集成到智能眼镜和陪伴娃娃中，以便用户可以体验到近乎实时的智能助手交互。

值得一提的是，GLM-Realtime 进一步支持 Function Call 功能。不仅能够依靠自身的知识和能力，还能灵活调用外部知识和工具，从而能够拓展到更广泛的商业场景。

👤 用户并发权益

API 调用会受到速率限制，当前我们限制的维度是请求并发数量（在途请求任务数量）。不同等级的用户并发保障如下。

V0	V1	V2	V3
5	10	15	20

接口参数

音视频实时 API（通过 `/realtime`）构建在 WebSocket API 之上。

API 请求地址: `wss://open.bigmodel.cn/api/paas/v4/realtime`

请求头

参数名称	类型	必填	参数描述
Authorization	String	是	鉴权信息: JWT 或 API Key

公共参数

参数名称	类型	参数描述
event_id	String	由客户端生成的id, 用于标识此事件
type	String	事件类型
client_timestamp	Integer	调用端发起调用的时间戳, 毫秒

VAD 检测

Realtime API支持两种VAD检测方式, 根据参数 `turn_detection.type` 控制。

1. Server VAD模式, 模型智能检测
2. Client VAD模式, 客户端自行决定触发模型推理时机

	Server VAD 模式	客户端 VAD 模式
对应字段	server_vad	client_vad
客户端逻辑复杂度	低, 仅需不停的上传音频	高, 需判断上传时机, 和触发模型时机
打断	由 Realtime Server 完全托管	由客户端自行决定
说话检测	由 Realtime Server 判断	由客户端自行判断

事件时序

(基本对话流程) 响应阶段, 不同类型的事件之间没有顺序关系(单个类型事件保证有序),在 websocket 通道中流式输出

以 `client vad` 视频通话为例事件流如下：

>

业务方

realtime api

建立 ws 链接

session.created 返回会话基本信息

session.update 设置会话信息

session.updated 会话信息已设置

音视频数据传输阶段

input_audio_buffer.append 追加音频

input_audio_buffer.append 流式追加音频

input_audio_buffer.append_video_frame 追加视频帧(video mode)

input_audio_buffer.commit 提交音频

input_audio_buffer.committed 收到提交音频

conversation.created 会话创建

响应生成阶段

response.create 触发生成回复

response.created 回复开始生成

rate_limited.updated 速率限制

conversation.item.input_audio_transcription.completed 音频转写完成

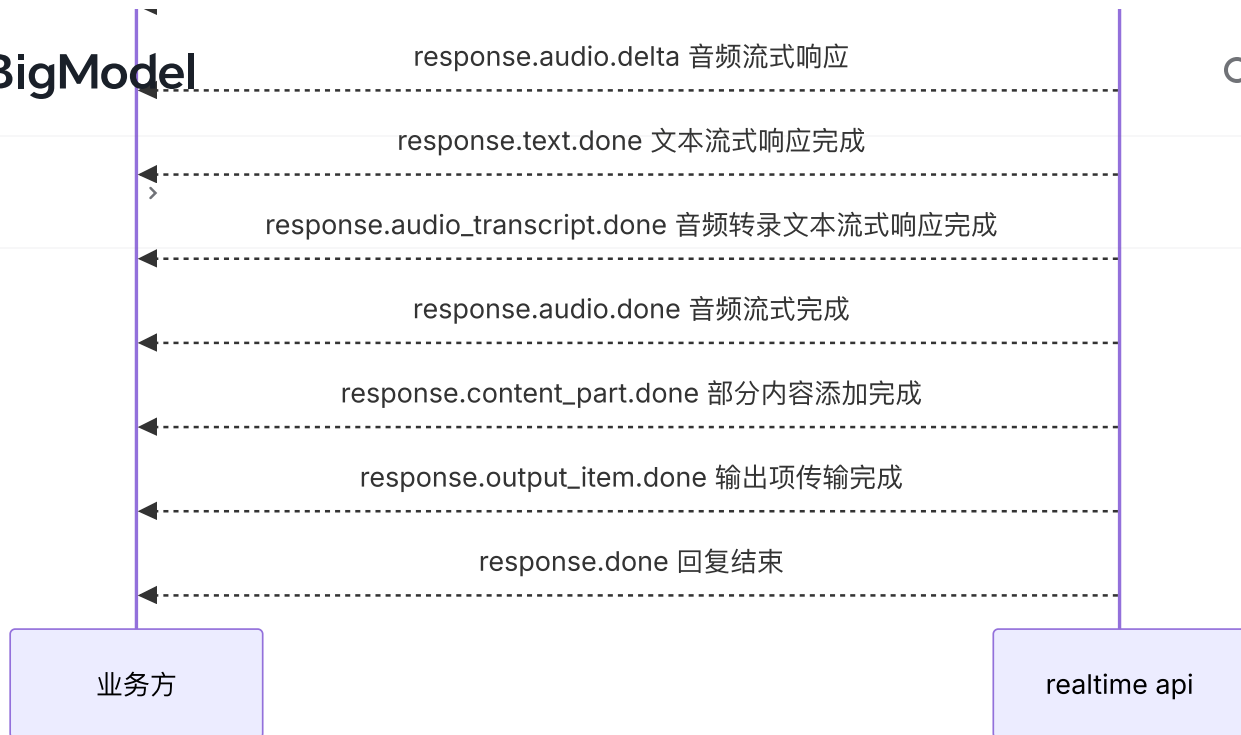
response.output_item.added 输出项已添加

conversation.item.created 会话项创建完成

response.content_part.added 部分内容已添加

response.text.delta 文本流式响应

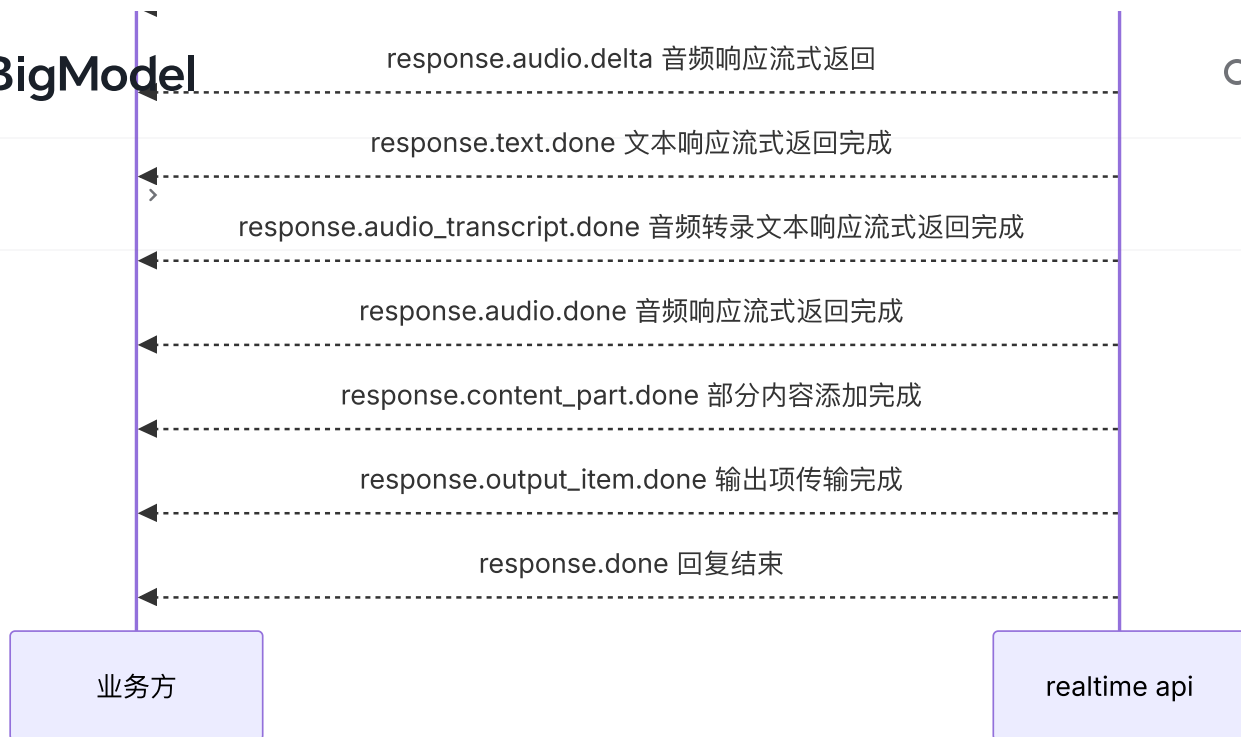
response.audio_transcript.delta 音频转录文本流式响应



Server VAD

以 `server vad` 视频通话为例事件流如下：





Function call

以 `client vad` 语音通话为例事件流如下：

业务方

realtime api

建立ws 链接

session.created 返回会话基本信息

session.update 设置会话信息,更新 tools

session.updated 会话信息已设置

input_audio_buffer.append 追加音频

input_audio_buffer.append 流式追加音频

input_audio_buffer.commit 提交音频 (VAD 结束)

input_audio_buffer.committed 收到提交音频

response.create 触发生成回复

conversation.created 会话创建

response.created 回复开始生成

rate_limit.updated 速率限制

conversation.item.created 会话项创建完成

response.output_item.added 输出项已添加

response.content_part.added 部分内容已添加

response.function_call_arguments.done 收到工具调用信息

response.content_part.done 部分内容已完成

response.output_item.done 输出项已完成

response.done 回复结束

工具调用阶段

conversation.item.create 上报工具调用结果

response.create 触发模型继续生成

>



数据结构

RealtimeConversationItem

- **用途:** 定义对话中的项，可以是消息、函数调用或函数调用响应。
- **属性:**
- **id** (string, 可选): 项的唯一 ID，可以由客户端生成。
- **type** (string, 必需): 项的类型 (`message` , `function_call` , `function_call_output`)。
- **object** (string, 必需): 始终为 `"realtime.item"` 。
- **status** (string, 可选): 项的状态 (`completed` , `incomplete`)。
- **role** (string, 可选): 消息发送者的角色 (`user` , `assistant` , `system`)，仅在 `message` 类型时适用。
- **content** (array, 可选): 消息内容数组。
- **type** (string, 必需): 内容类型 (`input_audio` , `input_text` , `text`)。



`text` (string, 可选): 文本内容。

`audio` (string, 可选): Base64 编码的音频数据。



- `transcript` (string, 可选): 音频的转录文本。
- `name` (string, 可选): 函数调用的名称, 用于 `function_call` 类型。
- `arguments` (string, 可选): 函数调用的参数, 用于 `function_call` 类型。
- `output` (string, 可选): 函数调用的输出, 用于 `function_call_output` 类型。

RealtimeResponse

- **用途:** 定义服务器返回的响应对象结构。
- **属性:**
 - `id` (string, 必需): 响应的唯一 ID。
 - `object` (string, 必需): 始终为 `"realtime.response"`。
 - `status` (string, 必需): 响应的状态 (`completed` , `cancelled` ,)。
 - `usage` (object, 可选): 响应的使用统计信息, 对应于计费信息。暂时都返回 0, 实际计算规划开发中
 - `total_tokens` (integer, 可选): 总共使用的令牌数量。
 - `input_tokens` (integer, 可选): 输入令牌数量。
 - `output_tokens` (integer, 可选): 输出令牌数量。
 - `input_token_details` (object, 可选): 关于输入令牌的详细信息。
 - `cached_tokens` (integer, 可选): 使用缓存令牌的数量
 - `text_tokens` (integer, 可选): 使用文本令牌的数量。
 - `audio_tokens` (integer, 可选): 使用音频令牌的数量。
 - `output_token_details` (object, 可选): 关于输出令牌的详细信息。
 - `text_tokens` (integer, 可选): 输出的文本令牌数量。
 - `audio_tokens` (integer, 可选): 输出的音频令牌数量。

客户端事件

事件	说明
RealtimeClientEventSessionUpdate	会话配置，通过此事件更新会话的默认配置
RealtimeClientEventTranscriptionSessionUpdate	转录会话配置，发送此事件以更新转录会话
RealtimeClientEventInputAudioBufferAppend	上传音频
RealtimeClientEventInputAudioBufferAppendVideoFrame	视频通话模式时，上报视频帧
RealtimeClientEventInputAudioBufferCommit	提交音频
RealtimeClientEventInputAudioBufferClear	清除缓冲区中的音频
RealtimeClientEventConversationItemCreate	用于文本输入以及上传function call的结果
RealtimeClientEventConversationItemDelete	删除会话历史中的某一轮对话事项
RealtimeClientEventConversationItemRetrieve	查看会话历史中的某一轮对话事项
RealtimeClientEventResponseCreate	创建模型调用，推理回复
RealtimeClientEventResponseCancel	取消模型调用

RealtimeClientEventSessionUpdate

通过此事件更新会话的默认配置，默认为 `client vad` 下的语音通话，并且会使用上面参数的默认值，比如 `output_audio_format` 为 `pcm`。

特殊说明：当 `session.update` 切换 `chat_mode` 通话模式时，会有系统默认的对话历史处理策略：

- 从 `video_passive` 到 `audio`，对话历史会丢弃；
- 从 `audio` 到 `video_passive`，对话历史会保留；

参数名称	类型	参数描述	是否必填
event_id	string	事件ID，客户端自行生成	N
client_timestamp	Integer	调用端发起调用的时间戳，毫秒	N
session	object	实时对话的配置信息	Y

参数名称	类型	参数描述	是否必填
type	string	事件类型，会话配置的事件类型为 <code>session.update</code>	Y

实时对话的 `session` 对象参数说明:

参数名称	类型	参数描述
model	string	模型名，默认值: <code>glm-realtime</code> - 9B模型: <code>glm-realtime-flash</code> - 32B模型: <code>glm-realtime-air</code>
modalities	string	控制模型是否输出文本或音频，默认值: <code>["text", "audio"]</code>
instructions	string	系统指令，用于引导模型生成期望的响应。默认内容见下
voice	string	音色。目前音色如下: 1. 通用男声 <code>xiaochen</code> 2. 通用女声 <code>tongtong</code> (默认) 3. 甜美女性 <code>female-tianmei</code> 4. 少女 <code>female-shaonv</code> 5. 青年大学生 <code>male-qn-daxuesheng</code> 6. 精英青年 <code>male-qn-jingying</code> 7. 萌萌女童 <code>lovely_girl</code>
input_audio_format	string	音频输入格式，支持wav和pcm；输入PCM的话最好带上
output_audio_format	string	音频输出格式。当前仅支持取值"pcm"，采样率24 kHz，
input_audio_noise_reduction	object	输入音频降噪配置，结构见下表。
turn_detection	object	vad类型，不传表示client vad，结构见下表。
temperature	float	模型温度，取值范围 <code>[0.0, 1.0]</code> ，值越大，会使输出更随
max_response_output_tokens	string	回复的最大长度，对应文本token计数，取值范围 <code>(0, 1024]</code>
tools	<code>List<Tool></code>	工具定义触发 <code>Function Call</code> ，目前只支持语音通话，
beta_fields	object	自定义字段，结构见下表。

`input_audio_noise_reduction` 对象参数说明:





BigModel

数据智能

	类型	参数描述	🔍	⋮
type	string	降噪类型。near_field 适用于近距离说话的麦克风，如耳机；far_fie		

vad_turn_detection 对象参数说明:

参数名称	类型	参数描述
type	string	VAD检测的类型，有且仅能填写server_vad
create_response	bool	当VAD停止事件发生时，是否自动生成响应
interrupt_response	bool	当VAD启动事件发生时，是否自动中断任何正在进行的响应，并将转接给下一个说话人

Tool 对象参数说明:

参数名称	类型	参数描述
type	string	工具的类型，设置为function
name	string	函数名称
description	string	用于描述函数功能。模型会根据这段描述决定函数调用方式。
parameters	object	parameters字段需要传入一个 Json Schema 对象，以准确地定义函数的参数。

beta_fields 对象参数说明:



参数配置

参数名称	类型	参数描述	是否必填
chat_mode	string	通话模式： <code>video_passive</code> 、 <code>audio</code> （默认）	Y
tts_source	string	语音转文字的方式，支持：e2e。	N
auto_search	bool	是否开启网页检索(true表示在服务端内置搜索工具)	N
greeting_config	object	开场白(或欢迎语)设置，AI首先说话时使用	N

`greeting_config` 对象参数说明:

参数名称	类型	参数描述	是否必填
enable	bool	是否启用开场白(或欢迎语)	N
content	string	开场白(或欢迎语)自定义内容，不超过1024个字符	N

`instructions` 默认指令

模式	对应参数	指令内容
语音通话	<code>chat_mode</code> 为 <code>audio</code>	你是一个名为小智的人工智能助手，是基于 GLM-4o 模型开发的
视频通话	<code>chat_mode</code> 为 <code>video</code>	你是一个名叫小智的人工智能助手，基于智谱AI 的 GLM 模型开发

`session.update` 消息事件发送示例:

```
{
  "event_id": "6357c85e-fee5-41e8-8da4-01ad2593c07f",
  "client_timestamp": 1751955406660,
  "session": {
    "model": "glm-realtime",
    "modalities": ["audio", "text"],
    "instructions": "You are a helpful, witty, and friendly AI. Act like a human.",
    "voice": "tongtong",
    "input_audio_format": "wav",
    "output_audio_format": "pcm",
    "input_audio_noise_reduction": {
      "type": "far_field"
    },
    "tools": [
      {
        "type": "function",
        "name": "get_avg_temp",
        "description": "Get the current weather conditions at the specified location.",
        "parameters": {
          "type": "object",
          "properties": {
            "city": {
              "type": "string",
              "description": "Name of city"
            }
          },
          "required": ["city"]
        }
      }
    ],
    "temperature": 0.7,
    "max_response_output_tokens": "inf",
    "beta_fields": {
      "chat_mode": "audio",
      "tts_source": "e2e",
      "auto_search": true,
      "greeting_config": {
        "enable": true,
        "content": "你好，我是小智，有什么可以帮助你吗？"
      }
    }
  }
}
```

```
}  
  "type": "session.update"  
}
```

RealtimeClientEventTranscriptionSessionUpdate

转录会话配置，发送 `transcription_session.update` 事件以更新转录会话。

参数名称	类型	参数描述
event_id	string	事件ID，客户端自行生成
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
session	object	实时对话的配置信息
type	string	事件类型，会话配置的事件类型为 <code>transcription_session.updat</code>

`session` 对象参数说明:

参数名称	类型	参数描述
input_audio_format	string	输入音频格式，目前支持 <code>pcm</code> 和 <code>wav</code> 格式
input_audio_noise_reduction	object	输入音频降噪配置，目前支持 <code>far_field</code> 和 <code>near_field</code>
modalities	array	实时对话的模态，目前支持 <code>text</code> 和 <code>audio</code> 两种模态，黑
turn_detection	object	<code>ClientVAD</code> 客户端必须手动触发模型响应。 <code>ServerVAD</code>

`turn_detection` 对象参数说明:

参数名称	类型	参数描述
create_response	boolean	是否在 VAD 停止事件发生时自动生成响应。不适用于转录会话。

参数名称	类型	参数描述
interrupt_response	boolean	当 VAD 开始事件发生时，是否自动中断任何正在进行的响应，并输
prefix_padding_ms	Integer	仅用于 ServerVAD 模式。在VAD检测到语音之前要包含的音频量
silence_duration_ms	Integer	仅用于 ServerVAD 模式。用于检测语音停止的静音持续时间（以毫
threshold	float	仅用于 ServerVAD 模式。VAD的激活阈值（0.0 到 1.0），默认为 C

```
{
  "event_id": "7b6aab70-751e-4270-9461-18709a1cb036",
  "client_timestamp": 1751620391884,
  "type": "transcription_session.update",
  "session": {
    "input_audio_format": "pcm",
    "input_audio_noise_reduction": {
      "type": "far_field"
    },
    "modalities": ["text", "audio"],
    "turn_detection": {
      "type": "server_vad",
      "threshold": 0.5,
      "prefix_padding_ms": 300,
      "silence_duration_ms": 500,
      "create_response": true,
      "interrupt_response": true
    }
  }
}
```

RealtimeClientEventInputAudioBufferAppend

此事件用于上传音频流至缓冲区。

- Server VAD 模式将由模型自动检测语音并决定何时提交；
- Client VAD 模式需要手动上传并提交音频。上传时可以自行决定音频长度，音频越短响应时间越快，最长可上传30秒；



3. 音频发送的最高速率为 50QPS，超过后会被限流丢弃，实时音频流推荐按 100ms 一帧切分，每秒发送 10 帧



参数名称	类型	参数描述
event_id	string	事件ID，客户端自行生成
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
audio	string	音频(wav or pcm)二进制的 base64 编码字符串
type	string	事件类型，上传音频流的事件类型为 <code>input_audio_buffer.append</code>

`input_audio_buffer.append` 消息事件发送示例:

```
{
  "event_id": "7b6aab70-751e-4270-9461-18709a1cb036",
  "client_timestamp": 1751620391884,
  "audio": "UklGRiQZAABXQVZFZm10IBAAAAABAAEAgD4AAAB9AAACABAAZGF0YQAZAAAR9Hrx..."
  "type": "input_audio_buffer.append"
}
```

RealtimeClientEventInputAudioBufferAppendVideoFrame

此事件用于上传视频帧至缓冲区。当前版本下，`chat_mode` 为 `video_passive` 的视频帧均随音频同时发送，ServerVAD 模式下会自动跟随音频上传，CliendVAD 模式下需要按照指定的 fps 向服务端推送 base64 编码的 jpg 图片。

参数名称	类型	参数描述
event_id	string	事件ID，客户端自行生成
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
type	string	事件类型，上传视频帧的事件类型为 <code>input_audio_buffer.append</code>
video_frame	string	支持 base64 编码的 jpg 格式图片

`input_audio_buffer.append_video_frame` 消息事件发送示例：

```
{
  "event_id": "53915927-1618-430c-8423-236a915348e1",
  "client_timestamp": 1751857813096,
  "video_frame": "/9j/2wCEAAgGBgcGBQgHBwcJCQgKDBQNDAsLDBkSEw8UHRofHh0aHBwgJC4nIC0",
  "type": "input_audio_buffer.append_video_frame"
}
```

RealtimeClientEventInputAudioBufferCommit

提交已经上传的音频文件，此事件前必须进行 `input_audio_buffer.append`，且必须上传一个有效音频或视频文件，否则提交事件会报错。ServerVAD 模式下不需要发送此事件，模型将自动上传并提交音频。

调用 `input_audio_buffer.commit` 时，如果缓冲区内发过 `video_frame`，会一起打包提交调用模型推理。

参数名称	类型	参数描述
event_id	string	事件ID，客户端自行生成
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
type	string	事件类型，上传音视频的事件类型为 <code>input_audio_buffer.commit</code>

```
{
  "event_id": "7ac0aba2-92a0-42e9-9d7b-86972c6b75ac",
  "client_timestamp": 1751858272957,
  "type": "input_audio_buffer.commit"
}
```

RealtimeClientEventInputAudioBufferClear

客户端发送 `input_audio_buffer.clear` 事件用于清除缓冲区中的音频数据, 服务端使用 `input_audio_buffer.cleared` 事件进行响应。

参数名称	类型	参数描述
event_id	string	事件ID, 客户端自行生成
client_timestamp	Integer	调用端发起调用的时间戳, 毫秒
type	string	事件类型, 清除上传音频的事件类型为 <code>input_audio_buffer.clea</code>

input_audio_buffer.clear 消息事件发送示例:

```
{
  "event_id": "7ac0aba2-92a0-42e9-9d7b-86972c6b75ac",
  "client_timestamp": 1751858272957,
  "type": "input_audio_buffer.clear"
}
```

RealtimeClientEventConversationItemCreate

向对话上下文中添加一个 item, 包含消息、函数调用响应结果, 可以将此部分结果放入对话历史 (session context/history)。如果传入文本为空或 function.call.item 为空时, 会发送一个错误事件;

参数名称	类型	参数描述
event_id	string	事件ID，客户端自行生成
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
item	RealtimeConversationItem	见数据结构 RealtimeConversationItem
type	string	事件类型，填充对话信息的事件类型为 conversation.item.create

conversation.item.create 消息事件发送示例(比如 function_call_output 类型):

```
{
  "event_id": "701360cc-5b4a-4c27-a632-266e825fff76",
  "client_timestamp": 1751594210037,
  "item": {
    "output": "{\\"status\\": \\"success\\", \\"message\\": \\"Average temperature of\\",
    "object": "realtime.item",
    "type": "function_call_output"
  },
  "type": "conversation.item.create"
}
```

RealtimeClientEventConversationItemDelete

向对话上下文中添加一个item，包含消息、函数调用响应结果，可以将此部分结果放入对话历史（session context/history）。如果传入文本为空或function.call.item为空时，会发送一个错误事件；

event_id	string	事件ID，客户端自行生成
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
type	string	事件类型，填充对话信息的事件类型为 <code>conversation.item.delete</code>
item_id	string	被删除的对话项的 <code>item_id</code> 。

`conversation.item.delete` 消息事件发送示例:

```
{
  "event_id": "701360cc-5b4a-4c27-a632-266e825fff76",
  "client_timestamp": 1751594210037,
  "item_id": "item3651646b143b4df8a9fc32a9dab574c7",
  "type": "conversation.item.delete"
}
```

RealtimeClientEventConversationItemRetrieve

参数名称	类型	参数描述
event_id	string	事件ID，客户端自行生成
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
type	string	事件类型，填充对话信息的事件类型为 <code>conversation.item.retri</code>
item_id	string	被检索的对话项的 <code>item_id</code> 。

`conversation.item.retrieve` 消息事件发送示例:

```
{
  "event_id": "rci_item5269108b10654d4480d614738291bfff",
  "client_timestamp": 1751962424281,
  "item_id": "item5269108b10654d4480d614738291bfff",
  "type": "conversation.item.retrieve"
}
```

RealtimeClientEventResponseCreate

此事件为创建服务器响应，同时也表示触发模型推理。ServerVAD模式服务器会自动创建响应，ClientVAD模式进行视频通话时，需以这个时间点的视频帧和音频传给模型；

当 `chat_mode` 为 `video` 时，提交事件之前必须通过 `input_audio_buffer.append_video_frame` 事件上传至少一张图片，否则无法创建模型回复，会返回 `video_model_query_error` 错误事件；

参数名称	类型	参数描述	
event_id	string	事件ID，客户端自行生成	N
client_timestamp	Integer	调用端发起调用的时间戳，毫秒	N
type	string	事件类型，触发模型推理的事件类型为 <code>response.create</code>	

`response.create` 消息事件发送示例：

```
{
  "event_id": "e0b458a4-8ae2-4cda-99e0-7686607aaa3c",
  "client_timestamp": 1751858272959,
  "type": "response.create"
}
```

RealtimeClientEventResponseCancel

此事件可取消正在进行的响应，服务器将响应一个 `response.cancelled` 事件，如果没有响应可取消，服务器将响应一个错误。

参数名	类型	参数描述	Q	:
event_id	string	事件ID，客户端自行生成		N
client_timestamp	Integer	调用端发起调用的时间戳，毫秒		N
type	string	事件类型，打断进行中的响应事件类型为 <code>response.cancel</code>		

`response.cancel` 消息事件发送示例：

```
{
  "event_id": "e0b458a4-8ae2-4cda-99e0-7686607aaa3c",
  "client_timestamp": 1751858272959,
  "type": "response.cancel"
}
```

服务端事件

事件	说明
<code>RealtimeServerEventError</code>	发生错误时的服务器事件
<code>RealtimeServerEventSessionCreated</code>	创建对话时的服务器事件，在创
<code>RealtimeServerEventSessionUpdated</code>	会话更新时服务器事件。
<code>RealtimeServerEventTranscriptionSessionUpdated</code>	转录会话更新时服务器事件。
<code>RealtimeServerEventConversationItemCreated</code>	创建对话时的服务器事件。
<code>RealtimeServerEventConversationItemDeleted</code>	响应 <code>conversation.item.dele</code>
<code>RealtimeServerEventConversationItemRetrieved</code>	响应 <code>conversation.item.retr</code>
<code>RealtimeServerEventConversationItemInputAudioTranscriptionCompleted</code>	启用了输入音频转文本并且转文
<code>RealtimeServerEventConversationItemInputAudioTranscriptionFailed</code>	启用了输入音频转文本并且转文
<code>RealtimeServerEventInputAudioBufferCommitted</code>	当输入音频缓冲区由客户端提交
<code>RealtimeServerEventInputAudioBufferCleared</code>	使用 <code>input_audio_buffer.clea</code>



事件

BigModel

	说明
<code>RealtimeServerEventInputAudioBufferSpeechStarted</code>	ServerVAD模式下检测到语音时
<code>RealtimeServerEventInputAudioBufferSpeechStopped</code>	ServerVAD模式下检测语音停止
<code>RealtimeServerEventResponseOutputItemAdded</code>	响应生成过程中创建新的对话项
<code>RealtimeServerEventResponseOutputItemDone</code>	输出项标记为 done 时发出的服务器事件。
<code>RealtimeServerEventResponseContentPartAdded</code>	响应生成期间将新的内容部分添加到响应中。
<code>RealtimeServerEventResponseContentPartDone</code>	当内容部分在助手消息项中完成时发出的服务器事件。
<code>RealtimeServerEventResponseFunctionCallArgumentsDone</code>	模型生成的函数调用参数完成流式处理时的服务器事件。
<code>RealtimeServerEventResponseFunctionCallSimpleBrowser</code>	视频链路触发了内置搜索的服务。
<code>RealtimeServerEventResponseTextDelta</code>	更新模型生成的文本时的服务器事件。
<code>RealtimeServerEventResponseTextDone</code>	模型生成的文本完成流式处理时的服务器事件。
<code>RealtimeServerEventResponseAudioTranscriptDelta</code>	更新模型生成的音频输出文本时的服务器事件。
<code>RealtimeServerEventResponseAudioTranscriptDone</code>	模型生成的音频输出文本完成流式处理时的服务器事件。
<code>RealtimeServerEventResponseAudioDelta</code>	更新模型生成的音频时的服务器事件。
<code>RealtimeServerEventResponseAudioDone</code>	模型生成的音频完成流式处理时的服务器事件。
<code>RealtimeServerEventResponseCreated</code>	创建新的响应时的服务器事件。
<code>RealtimeServerEventResponseCancelled</code>	对 <code>response.cancel</code> 事件的响应。
<code>RealtimeServerEventResponseDone</code>	响应完成流式处理时的服务器事件。
<code>RealtimeServerEventRateLimitsUpdated</code>	在响应开始时发出，以指示更新速率限制。
<code>RealtimeServerEventHeartbeat</code>	心跳保活的服务器事件。

RealtimeServerEventError

发生错误时，系统会返回服务器 `error` 事件（可能是客户端问题，也可能是服务器问题，具体可查看错误码文档）。大多数错误都是可恢复的，并且会话将保持打开状态。



参数

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>error</code> 。
error	object	错误的详细信息。

`error` 配置:

参数名称	类型	参数描述
type	string	错误类型。
code	string	错误代码。
message	string	错误消息。

`error` 消息事件响应示例:

```
{
  "event_id": "event_890",
  "type": "error",
  "error": {
    "type": "invalid_request_error",
    "code": "invalid_event",
    "message": "The 'type' field is missing."
  }
}
```

RealtimeServerEventSessionCreated

在创建会话后会立即返回服务器 `session.created` 事件

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id

参数名称	类型	参数描述
type	string	事件类型必须是 <code>session.created</code>
session	object	当前会话下的配置信息。

`session.created` 消息事件响应示例:

```
{
  "event_id": "event5ad8cd18a8d544e59c581dcd7b1912d5",
  "type": "session.created",
  "client_timestamp": 1751868138242,
  "session": {
    "object": "realtime.session",
    "id": "20250707140217dc3ddb78460c420b",
    "model": "glm-realtime",
    "modalities": ["text", "audio"],
    "voice": "default",
    "input_audio_format": "wav",
    "output_audio_format": "pcm",
    "temperature": 0.05,
    "beta_fields": {
      "chat_mode": "audio"
    }
  }
}
```

RealtimeServerEventSessionUpdated

更新会话后会立即返回服务器 `session.updated` 事件

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>session.updated</code>
session	object	当前会话下的配置信息。

>

```
"event_id": "event102c4efa9344b24a274e1d1df2a17ec",
"type": "session.updated",
"client_timestamp": 1751868138291,
"session": {
  "object": "realtime.session",
  "id": "20250707140217dc3ddb78460c420b",
  "model": "glm-realtime",
  "modalities": ["text", "audio"],
  "instructions": "You are a helpful, witty, and friendly AI. Act like a human",
  "voice": "tongtong",
  "input_audio_format": "wav",
  "output_audio_format": "pcm",
  "input_audio_noise_reduction": {
    "type": "far_field"
  },
  "turn_detection": {
    "type": "server_vad",
    "create_response": true,
    "interrupt_response": true,
    "prefix_padding_ms": 300,
    "silence_duration_ms": 500,
    "threshold": 0.5
  },
  "tools": [
    {
      "name": "search_engine_auto",
      "description": "多功能网络搜索工具，旨在检索互联网上的实时、准确和全面的信息",
      "parameters": {
        "type": "object",
        "properties": {
          "q": {
            "type": "string",
            "description": "搜索查询"
          }
        }
      },
      "type": "function"
    }
  ]
},
],
```



```
"temperature": 0.7,
"max_response_output_tokens": "inf",
"beta_fields": {
  "chat_mode": "audio",
  "tts_source": "e2e",
  "auto_search": true
}
}
```

RealtimeServerEventTranscriptionSessionUpdated

客户端通过 `transcription_session.update` 更新转录会话后，系统会立即返回 `transcription.session.updated` 事件。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>transcription.session.updated</code>
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
session	object	当前会话下的配置信息。

`transcription.session.updated` 消息事件响应示例:

```
{
  "event_id": "event_5678",
  "type": "transcription_session.updated",
  "client_timestamp": 1751958821863,
  "session": {
    "id": "sesscf503d9060b04549b9768a591870e3a4",
    "object": "realtime.transcription_session",
    "input_audio_format": "pcm16",
    "input_audio_noise_reduction": {
      "type": "far_field"
    },
    "modalities": ["text", "audio"],
    "turn_detection": {
      "type": "server_vad",
      "threshold": 0.5,
      "prefix_padding_ms": 300,
      "silence_duration_ms": 500,
      "create_response": true,
      "interrupt_response": true
    }
  }
}
```

RealtimeServerEventConversationItemCreated

创建对话项时，将返回 `conversation.item.created` 服务器事件。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>conversation.item.created</code> 。
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
item	<code>RealtimeConversationItem</code>	见数据结构 <code>RealtimeConversationItem</code>

`conversation.item.created` 消息事件响应示例:

```
{
  "event_id": "event7eed01ee14f47b9a7e013aab1e6e243",
  "type": "conversation.item.created",
  "client_timestamp": 1751868140846,
  "item": {
    "content": [
      {
        "type": "input_audio"
      }
    ],
    "id": "item3651646b143b4df8a9fc32a9dab574c7",
    "object": "realtime.item",
    "role": "user",
    "status": "completed",
    "type": "message"
  }
}
```

RealtimeServerEventConversationItemDeleted

删除对话项时，将返回 `conversation.item.deleted` 服务器事件。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id。
type	string	事件类型必须是 <code>conversation.item.deleted</code> 。
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
item_id	string	被删除的对话项的 item_id。

`conversation.item.deleted` 消息事件响应示例:

```
{
  "event_id": "event7eed01ee14f47b9a7e013aab1e6e243",
  "type": "conversation.item.deleted",
  "client_timestamp": 1751868140846,
  "item_id": "item3651646b143b4df8a9fc32a9dab574c7"
}
```

RealtimeServerEventConversationItemRetrieved

检索对话项时，将返回 `conversation.item.retrieved` 服务器事件。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id。
type	string	事件类型必须是 <code>conversation.item.retrieved</code> 。
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
item	<code>RealtimeConversationItem</code>	见数据结构 <code>RealtimeConversationItem</code>

`conversation.item.retrieved` 消息事件响应示例:

```
{
  "event_id": "eventd35a0e7ae2204f468503aab2def5c5b0",
  "type": "conversation.item.retrieved",
  "client_timestamp": 1751962424320,
  "item": {
    "content": [
      {
        "text": "今天北京的天气怎么样? ",
        "type": "input_text"
      }
    ],
    "id": "item5269108b10654d4480d614738291bfff",
    "object": "realtime.item",
    "role": "user",
    "status": "completed",
    "type": "message"
  }
}
```

RealtimeServerEventConversationItemInputAudioTranscriptionCompleted

写入音频缓冲区的语音转文本的结果。语音转文本与响应创建异步运行，该事件可能发生在响应事件之前或者之后；

此部分转文本是独立模型，输出的内容可能和模型推理的结果有部分出入（也可能为空），转文本的结果仅作为参考，不作为输入到Realtime大模型中的具体结果。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>conversation.item.input_audio_transcriptio</code>
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
item_id	string	包含音频的用户消息项的 ID。
content_index	integer	包含音频的内容部分的索引。
transcript	string	语音转文本后的文本。

`conversation.item.input_audio_transcription.completed` 消息事件响应示例:

```
{
  "event_id": "event59cbcbd87d38444f9f933ae640421ab5",
  "type": "conversation.item.input_audio_transcription.completed",
  "client_timestamp": 1751868141962,
  "item_id": "item3651646b143b4df8a9fc32a9dab574c7",
  "content_index": 0,
  "transcript": "请提供中国去年一年的平均气温。"
}
```

RealtimeServerEventConversationItemInputAudioTranscriptionFailed

配置了输入音频听录并且用户消息的听录请求失败时，系统会返回服务器

`conversation.item.input_audio_transcription.failed` 事件。此事件是与其他 `error` 事件分开的，以便客户端能够识别相关项。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>conversation.item.input_audio_transcriptio</code>
client_timestamp	Integer	调用端发起调用的时间戳，毫秒

参数名称	类型	参数描述
item_id	string	包含音频的用户消息项的 ID。
content_index	integer	包含音频的内容部分的索引。
error	object	错误的详细信息。

`conversation.item.input_audio_transcription.failed` 消息事件响应示例:

```
{
  "content_index": 0,
  "error": {
    "code": "asr_no_result",
    "message": "asr.DoStreamAsrOnceV2 fail, err: <nil>",
    "type": "ASR_ERROR"
  },
  "event_id": "event52031230c8d54ae89b741f079563ad54",
  "item_id": "iteme11f576720274f15a3a442adcbc33e60",
  "type": "conversation.item.input_audio_transcription.failed"
}
```

RealtimeServerEventInputAudioBufferCommitted

输入音频缓冲区由客户端提交或在ServerVAD模式下自动提交时，系统会返回

`input_audio_buffer.committed` 服务器事件。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>input_audio_buffer.committed</code> 。
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
item_id	string	创建的用户消息项的 ID。

`input_audio_buffer.committed` 消息事件响应示例:

```
{
  "event_id": "event8f38062413d84b03a2c33659b6c01764",
  "type": "input_audio_buffer.committed",
  "client_timestamp": 1751868140842,
  "item_id": "item3651646b143b4df8a9fc32a9dab574c7"
}
```

RealtimeServerEventInputAudioBufferCleared

客户端使用 `input_audio_buffer.clear` 事件清除输入音频缓冲区时，系统会返回 `input_audio_buffer.cleared` 服务器事件。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>input_audio_buffer.cleared</code> 。
client_timestamp	Integer	调用端发起调用的时间戳，毫秒

`input_audio_buffer.cleared` 消息事件响应示例：

```
{
  "eventId": "event225bbc71e064a4cb06b5c66be048353",
  "type": "input_audio_buffer.cleared",
  "client_timestamp": 1751868140842
}
```

RealtimeServerEventInputAudioBufferSpeechStarted

ServerVAD模式在音频缓冲区中检测到语音时会返回 `input_audio_buffer.speech_started` 服务器事件。



event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>input_audio_buffer.speech_started</code> 。
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
audio_start_ms	integer	从会话开始到首次检测到语音时，所有音频写入缓冲区的毫秒数。这
item_id	string	语音开始时创建的用户消息项的ID。

`input_audio_buffer.speech_started` 消息事件响应示例:

```
{
  "event_id": "event7f9312e0f6154ffda322d636cc1c15b8",
  "type": "input_audio_buffer.speech_started",
  "client_timestamp": 1751868138389,
  "audio_start_ms": 600,
  "item_id": "item26ba3e9d0ebf4712b8af4e02dc3e750e"
}
```

RealtimeServerEventInputAudioBufferSpeechStopped

ServerVAD模式在音频缓冲区中检测到语音结束时会返回 `input_audio_buffer.speech_stopped` 服务器事件, 然后继续还发送一个 `conversation.item.created` 事件, 其中包含从音频缓冲区创建的用户消息项。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>input_audio_buffer.speech_stopped</code> 。
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
audio_end_ms	integer	从会话开始到语音停止的毫秒数。这对应于发送到模型的音频结束，
item_id	string	语音停止时创建的用户消息项的ID。

`input_audio_buffer.speech_stopped` 消息事件响应示例:

```
{
  "event_id": "event2e2d5e3079e44ec7b7af6db4668c876c",
  "type": "input_audio_buffer.speech_stopped",
  "client_timestamp": 1751868140842,
  "audio_end_ms": 3000,
  "item_id": "item26ba3e9d0ebf4712b8af4e02dc3e750e"
}
```

RealtimeServerEventResponseOutputItemAdded

在响应生成过程中创建新项时，系统会返回服务器 `response.output_item.added` 事件。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>response.output_item.added</code> 。
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
response_id	string	response事件的唯一id
output_index	integer	响应中的输出项的索引。
item	<code>RealtimeConversationItem</code>	见数据结构 <code>RealtimeConversationItem</code>

```
{
  "event_id": "event23b36122778446f2863134c0c6ef39bc",
  "type": "response.output_item.added",
  "client_timestamp": 1751880098011,
  "response_id": "respfd67e70249a44b2da68c6b7fda172eec",
  "output_index": 0,
  "item": {
    "content": [{}],
    "id": "item3c6944530c524f9496ff384f4a7926b5",
    "object": "realtime.item",
    "role": "assistant",
    "status": "in_progress",
    "type": "message"
  }
}
```

RealtimeServerEventResponseOutputItemDone

当项完成流式处理时，系统会返回服务器 `response.output_item.done` 事件, 即使响应中断、不完整或取消时，系统也会返回此事件。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>response.output_item.done</code> 。
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
response_id	string	response事件的唯一id
output_index	integer	响应中的输出项的索引。
item	<code>RealtimeConversationItem</code>	见数据结构 <code>RealtimeConversationItem</code>

`response.output_item.done` 消息事件响应示例:

```
{
  "event_id": "event25b340b7f1ea447aae32398acd808d15",
  "type": "response.output_item.done",
  "client_timestamp": 1751882567872,
  "response_id": "respd037f1f0a5014c9a8020728369b3245c",
  "output_index": 0,
  "item": {
    "content": [{}],
    "id": "item0927d7cbaf504c6ba0ac25586a011466",
    "object": "realtime.item",
    "role": "assistant",
    "status": "completed",
    "type": "message"
  }
}
```

RealtimeServerEventResponseContentPartAdded

在响应生成期间将新的内容部分添加到助手消息项时，系统会返回

`response.content_part.added` 事件。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>response.content_part.added</code> 。
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
response_id	string	response事件的唯一id
item_id	string	创建的用户消息项的 ID。
output_index	integer	响应中的输出项的索引。
content_index	integer	项内容数组中的内容部分的索引。
part	object	已添加的内容部分。

`response.content_part.added` 消息事件响应示例:

```
{
  "event_id": "eventa5b46f534b5446ef9535b598a5cc3b6c",
  "type": "response.content_part.added",
  "client_timestamp": 1751882570004,
  "response_id": "resp2394c354528542278e015dd8bd156206",
  "item_id": "itemdc4c357a7c18421f8daeb01f9f766ade",
  "output_index": 0,
  "content_index": 0,
  "part": {
    "type": "audio"
  }
}
```

RealtimeServerEventResponseContentPartDone

当内容部分完成流式处理时，系统会返回服务器 `response.content_part.done` 事件，即使响应中断、不完整或取消时，系统也会返回此事件。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>response.content_part.done</code> 。
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
response_id	string	response事件的唯一id
item_id	string	创建的用户消息项的 ID。
output_index	integer	响应中的输出项的索引。
content_index	integer	项内容数组中的内容部分的索引。
part	object	已添加的内容部分。

`response.content_part.done` 消息事件响应示例:

```
{
  "event_id": "event415317bda1c94ef4bb766525c7cecd00",
  "type": "response.content_part.done",
  "client_timestamp": 1751882574951,
  "response_id": "resp2394c354528542278e015dd8bd156206",
  "item_id": "itemdc4c357a7c18421f8daeb01f9f766ade",
  "output_index": 0,
  "content_index": 0,
  "part": {
    "type": "audio"
  }
}
```

RealtimeServerEventResponseFunctionCallArgumentsDone

模型生成的函数调用时，系统会返回 `response.function_call_arguments.done` 事件。

当发给模型的query需要调用多次function call时，可能会返回多个调用，比如提问“帮我搜一下北京、上海的天气”，模型会返回2次function call的结果，系统也会返回两次

`response.function_call_arguments.done` 事件。

当前仅支持响应成功时返回此事件，中断、不完整或取消时正在支持中。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>response.function_call_arguments.done</code> 。
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
response_id	string	response事件的唯一id
arguments	string	函数调用参数, json字符串格式，需自行解析
name	string	函数的名称

`response.function_call_arguments.done` 消息事件响应示例:

```
{
  "event_id": "event64399231934b4f4ea2ed5528a34e700d",
  "type": "response.function_call_arguments.done",
  "client_timestamp": 1751886463623,
  "response_id": "respc883e54c410c47eab071b6adb35780b0",
  "output_index": 0,
  "name": "get_avg_temp",
  "arguments": "{\\"country\\": \\"中国\\"}"
}
```

RealtimeServerEventResponseFunctionCallSimpleBrowser

视频通话链路内置了搜索的工具，当识别到用户的提问需要通过搜索获取外部数据时，会返回此事件。服务内部会自动调用搜索接口获取数据，获取搜索结果后会再次调用模型，获取到模型回复后继续流式返回数据。

此事件在 `response.created` 事件之后，在 `response.audio_transcript.delta` 之前，如搜索结果报错，会返回错误事件 `video_model_query_error`。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>response.function_call.simple_browser</code> 。
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
name	string	搜索工具名称
session	object	会话信息对象

`session` 对象说明:

参数名称	类型	参数描述
beta_fields	object	包含beta阶段功能的字段对象

参数名称	类型	参数描述
simple_browser	object	简易浏览器相关功能的字段对象

simple_browser 对象说明:

参数名称	类型	参数描述
description	string	描述信息, 包含拖延话术
search_meta	string	搜索的元数据信息
meta	string	附加的元数据信息
text_citation	string	文本引用信息

response.function_call.simple_browser 消息事件响应示例:

```
{
  "event_id": "event789f99f9cc89494f8e24d8dc9fec00ff",
  "type": "response.function_call.simple_browser",
  "client_timestamp": 1751857817277,
  "name": "simple_browser",
  "session": {
    "beta_fields": {
      "simple_browser": {
        "description": "好的, 我马上帮你查查2022年的平均气温! ",
        "search_meta": "",
        "meta": "",
        "text_citation": ""
      }
    }
  }
}
```


流式返回模型生成的文本时，系统会返回 `response.text.delta` 事件, 文本对应于助手消息项的 `text` 内容部分。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>response.text.delta</code> 。
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
response_id	string	response事件的唯一id
item_id	string	创建的模型回复对话项的 ID。
output_index	integer	响应中的输出项的索引。
content_index	integer	项内容数组中的内容部分的索引。
delta	object	模型流式输出的文本

`response.function_call.simple_browser` 消息事件响应示例:

```
{
  "event_id": "eventc75fb153c81e48b0b40550a355399c86",
  "type": "response.text.delta",
  "client_timestamp": 1751857819992,
  "response_id": "resp28c0386dbe984b78b611b9ff9d6b04aa",
  "item_id": "item8bacdb6a76584718987af27992e90316",
  "output_index": 0,
  "content_index": 0,
  "delta": "中国的"
}
```

RealtimeServerEventResponseTextDone

当模型生成的文本完成流式处理时，系统会返回 `response.text.done` 事件。 文本对应于助手消息项的 `text` 内容部分，当响应中断、不完整或取消时，系统也会返回此事件。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>response.text.done</code>
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
response_id	string	response事件的唯一id
item_id	string	创建的模型回复对话项的 ID。
output_index	integer	响应中的输出项的索引。
content_index	integer	项内容数组中的内容部分的索引。
text	string	模型输出的最终完整文本。

`response.text.done` 消息事件响应示例:

```
{
  "event_id": "eventa10c7bf1ae3a43878ac70a0ae62b3c4b",
  "type": "response.text.done",
  "client_timestamp": 1751857822252,
  "response_id": "resp28c0386dbe984b78b611b9ff9d6b04aa",
  "item_id": "item8bacdb6a76584718987af27992e90316",
  "output_index": 0,
  "content_index": 0,
  "text": "2022年，中国的平均气温为10.51摄氏度，较常年偏高0.62摄氏度，为1961年以来仅次于20"
}
```

RealtimeServerEventResponseAudioTranscriptDelta

流式返回模型生成的音频输出语音转文本时，系统会返回 `response.audio_transcript.delta` 事件。此部分转文本是独立模型，输出的内容可能和模型推理的结果有部分出入（也可能为空），转文本的结果仅作为参考，不作为输入到Realtime大模型中的具体结果，建议不要将此事件作为后续事件的依赖项。

	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>response.audio_transcript.delta</code>
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
response_id	string	response事件的唯一id
item_id	string	创建的模型回复对话项的 ID。
output_index	integer	响应中的输出项的索引。
content_index	integer	项内容数组中的内容部分的索引。
delta	string	模型输出的语音转文本的结果。

`response.audio_transcript.delta` 消息事件响应示例:

```
{
  "event_id": "event2f132bfe1e14448e92235faf355cf958",
  "type": "response.audio_transcript.delta",
  "client_timestamp": 1751857820103,
  "response_id": "resp28c0386dbe984b78b611b9ff9d6b04aa",
  "item_id": "item8bacdb6a76584718987af27992e90316",
  "output_index": 0,
  "content_index": 0,
  "delta": "中国的"
}
```

RealtimeServerEventResponseAudioTranscriptDone

模型生成的音频输出听录完成流式处理时，系统会返回服务器

`response.audio_transcript.done` 事件。

当响应中断、不完整或取消时，系统也会返回此事件。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>response.audio_transcript.delta</code>
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
response_id	string	response事件的唯一id
item_id	string	创建的模型回复对话项的 ID。
output_index	integer	响应中的输出项的索引。
content_index	integer	项内容数组中的内容部分的索引。
transcript	string	模型输出的语音转文本的最终完整结果。

- 示例

```
{
  "event_id": "eventd86c702bcd30488f801ee171c4879fd9",
  "type": "response.audio_transcript.done",
  "client_timestamp": 175185782252,
  "response_id": "resp28c0386dbe984b78b611b9ff9d6b04aa",
  "item_id": "item8bacdb6a76584718987af27992e90316",
  "output_index": 0,
  "content_index": 0,
  "transcript": "2022年，中国的平均气温为10.51摄氏度，较常年偏高0.62摄氏度，为1961年以来1"
}
```

RealtimeServerEventResponseAudioDelta

流式返回模型生成的音频时，系统将返回 `response.audio.delta` 事件。delta是一个 `pcm` 格式 `base64` 编码的音频块。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id

参数名称	类型	参数描述
type	string	事件类型必须是 <code>response.audio.delta</code> 。
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
response_id	string	response事件的唯一id
item_id	string	创建的模型回复对话项的 ID。
output_index	integer	响应中的输出项的索引。
content_index	integer	项内容数组中的内容部分的索引。
delta	string	<code>base64</code> 编码的音频数据。

`response.audio.delta` 消息事件响应示例:

```
{
  "event_id": "event7e1636069ac84c50a14a653e441af170",
  "type": "response.audio.delta",
  "client_timestamp": 1751857822985,
  "response_id": "resp28c0386dbe984b78b611b9ff9d6b04aa",
  "item_id": "item8bacdb6a76584718987af27992e90316",
  "output_index": 0,
  "content_index": 0,
  "delta": "ev4f/gD+KP5b/n/+gf52/nz+lv7H/ur+Bv/x/uf+Fv9u/6T/iv9Z/2P/rf/7/xsA..."
}
```

RealtimeServerEventResponseAudioDone

当模型生成的音频完成流式处理时，系统将返回 `response.audio.done` 事件，当响应中断、不完整或取消时，系统也会返回此事件。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>response.audio.done</code>

参数名称	类型	参数描述
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
response_id	string	response事件的唯一id
item_id	string	创建的模型回复对话项的 ID。
output_index	integer	响应中的输出项的索引。
content_index	integer	项内容数组中的内容部分的索引。

`response.audio.done` 消息事件响应示例:

```
{
  "event_id": "eventadddd89e97b7e47e994e46702abd82622",
  "type": "response.audio.done",
  "client_timestamp": 1751857823000,
  "response_id": "resp28c0386dbe984b78b611b9ff9d6b04aa",
  "item_id": "item8bacdb6a76584718987af27992e90316",
  "output_index": 0,
  "content_index": 0
}
```

RealtimeServerEventResponseCreated

创建新响应时系统会返回 `response.created` 事件。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>response.created</code>
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
response	<code>RealtimeResponse</code>	见数据结构 <code>RealtimeResponse</code>

`response.created` 消息事件响应示例:

```
{
  "event_id": "event1eee3f0bd7a4424b8209a581d0c65ec2",
  "type": "response.created",
  "client_timestamp": 1751955411748,
  "response": {
    "object": "realtime.response",
    "id": "resp847c2c7fe6284c8f9381fbb3e0358597",
    "status": "in_progress"
  }
}
```

RealtimeServerEventResponseCancelled

当响应被取消时，系统会返回 `response.cancelled` 事件, 对客户端 `response.cancel` 事件的响应，如果存在正在进行中的response，如果没有正在进行中的response，会返回 `stop_task_error`。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>response.cancelled</code>
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
response	<code>RealtimeResponse</code>	见数据结构 <code>RealtimeResponse</code>

`response.cancelled` 消息事件响应示例:

```
{
  "event_id": "event1eee3f0bd7a4424b8209a581d0c65ec2",
  "type": "response.cancelled",
  "client_timestamp": 1751955411748,
  "response": {
    "object": "realtime.response",
    "id": "resp847c2c7fe6284c8f9381fbb3e0358597",
    "status": "cancelled"
  }
}
```

RealtimeServerEventResponseDone

当一轮对话回复结束，系统会返回 `response.done` 事件，无论最终状态如何，始终发出此事件，消耗的tokens会在该事件中返回。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>response.done</code>
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
response	<code>RealtimeResponse</code>	见数据结构 <code>RealtimeResponse</code>

`response.done` 消息事件响应示例:


```
{
  "event_id": "event87d8183824bb4a15a50d840105560192",
  "type": "response.done",
  "client_timestamp": 1751955413933,
  "response": {
    "object": "realtime.response",
    "id": "resp847c2c7fe6284c8f9381fbb3e0358597",
    "status": "completed",
    "usage": {
      "total_tokens": 788,
      "input_tokens": 750,
      "output_tokens": 38,
      "input_token_details": {
        "text_tokens": 350,
        "audio_tokens": 400
      },
      "output_token_details": {
        "text_tokens": 18,
        "audio_tokens": 20
      }
    }
  }
}
```

RealtimeServerEventRateLimitsUpdated

在响应开始时发出，以指示更新的速率限制。当创建响应时，一些令牌将被“预留”用于输出令牌，此处显示的速率限制反映了这种预留，一旦响应完成，将相应地进行调整。

参数名称	类型	参数描述
event_id	string	服务器事件的唯一id
type	string	事件类型必须是 <code>rate_limits.updated</code>
client_timestamp	Integer	调用端发起调用的时间戳，毫秒
rate_limits	object	速率限制信息列表。

参数名称	类型	参数描述
name	string	速率限制名称
limit	Integer	速率限制的最大值
remaining	Integer	达到限制前的剩余值
reset_seconds	Integer	距离速率限制重置还剩多少秒

rate_limits.updated 消息事件响应示例:

```
{
  "event_id": "eventb7dd4ecbdfd84634a63d1d2c22555c2b",
  "type": "rate_limits.updated",
  "client_timestamp": 1751955406776,
  "rate_limits": [
    {
      "name": "requests",
      "limit": 5,
      "remaining": 4,
      "reset_seconds": 1.0
    }
  ]
}
```

RealtimeServerEventHeartbeat

当会话创建/更新时会返回，后续每30s返回一次， heartbeat 表示对话当前是活跃的连接状态。



	类型	参数描述	🔍	⋮
event_id	string	服务器事件的唯一id		
type	string	事件类型必须是 heartbeat		
client_timestamp	Integer	调用端发起调用的时间戳，毫秒		

```
{
  "event_id": "eventa44ba7b0455547ecb79d5bb50ed858f0",
  "type": "heartbeat",
  "client_timestamp": 1751858270463
}
```